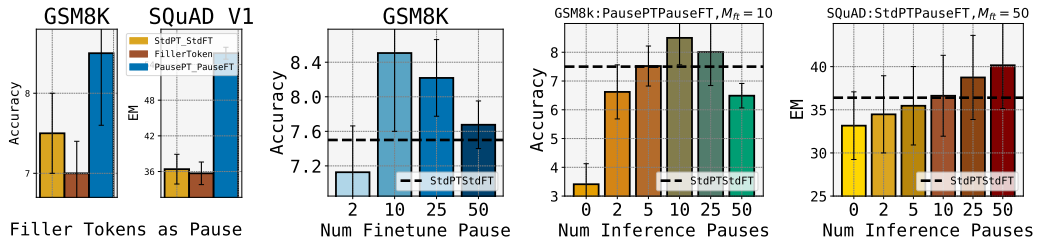

(a) (b) (c) (d)

Figure 4: **Key Ablations (§5):** (a) We compare a pause-*trained* model vs. a standard model delayed using a filler periods ('...'). As in Lanham et al. (2023), the filler periods do not give any gains. (b) There exists an optimal number of finetuning <pause> tokens (M_{ft}) for a given downstream dataset beyond which gains diminish. (c) and (d) We test the robustness of pause-trained models to varying number of inference time <pause> tokens (setting M_{inf} not equal to M_{ft}), which exposes the model to a serious test-time distribution shift. Pause-training degrades gracefully to shifts as wide as $M_{\text{inf}} \in [5, 25]$ for $M_{\text{ft}} = 10$ both for (c) PausePT_PauseFT and (d) StdPT_PauseFT.

Thus, to the core question of our exploration — whether delays help, hurt or do nothing at all — we find that the answer depends on when these delays are introduced. Specifically, pause-*pre*training appears crucial for delays to help in downstream inference-time. We conjecture that a standard-pretrained model has strong biases that prevent it from fully realizing the benefits of inference-time delays e.g., standard pretraining biases the model to be "quick" in its computations.

Remark: As a concluding note, we remind the reader that the PausePT_PauseFT model has a (deliberately injected) computational advantage compared to StdPT_StdFT, during finetuning and inference. However, there is no computational advantage during pause-pretraining since we equalize the number of tokens seen. In fact, this only results in a slight statistical disadvantage: the pause-pretrained model sees only 90% of the (meaningful) pretraining tokens that the standard model sees, as the remaining 10% are dummy <pause> tokens.

5 Ablations: Where and how many <pause> tokens to use

Number of <pause> tokens during finetuning Recall that we append M_{ft} copies of (the same) <pause> token to the prefix during finetuning. We find that for each downstream dataset, there is a corresponding optimal value of M_{ft}. For example, on GSM8k, 10 <pause> tokens are optimal with accuracy reducing to that of baseline as <pause> tokens are increased to 50 (See Figure 4b), while for SQuAD , 10 is sub-optimal (see Appendix E). Possibly, for each dataset, there exists a certain threshold of <pause> tokens beyond which the self-attention mechanism becomes overwhelmed.

Robustness to a varying number of inference-time pauses So far, we have set the inference-time delay to be the same as what was seen during finetuning ($M_{\text{inf}} = M_{\text{ft}}$). Next, we examine what happens if we vary M_{inf} during inference. Note that this presents a severe test-time distribution shift as we provide no supervision for the model until the last <pause> token (the $M_{\text{ft}}^{\text{th}}$ one) is seen. Thus the model may very well output garbage if we begin eliciting a response that is either premature ($M_{\text{inf}} < M_{\text{ft}}$) or belated ($M_{\text{inf}} > M_{\text{ft}}$). Yet, in practice, we find a (somewhat) graceful behavior.

First, PausePT_PauseFT model is robust to a wide test-time shift in the number of <pause> tokens (see Figure 4c and Appendix F): the performance remains above the baseline even if the inference-time pause tokens are half that of training-time. This is desirable in case of real-time fluctuations in computational constraints. Relatively, the StdPT_PauseFT model (wherein we inject delays only during finetuning) is even more more robust (see Fig 4d and also Appendix F).

Now, an ideal robustness criterion would be that, in the absence of *any* <pause> tokens, the pause-finetuned model performs just as well as a standard-finetuned model. Unfortunately, this isn't the case for any of our models. In fact, for PausePT_PauseFT, providing zero delay during inference

breaks its performance spectacularly (Figure 4c and also Appendix F), even if the model behaves reasonably with as few as 2 inference-time <pause> tokens. The design of *zero-delay-robust* pause-trained models is thus an important question for future work.

APPENDING VS. PREPENDING PAUSES In our main experiments, we chose to append <pause> tokens since it is the most natural format for a general setting e.g., in long-text-generation as in a conversational agent, one would append <pause> tokens to the current text rather than deploying the tokens all at once at the beginning of the conversation. Furthermore, when there is unidirectional attention, prepending these tokens should make no difference. Nevertheless, in our downstream tasks which use bidirectional attention on the prefix, it makes sense to consider prepending <pause> tokens. We investigate this in Table 2 in Appendix C. Most importantly, we find that, for PausePT_PauseFT, even prepending the <pause> token performs improves over standard end-to-end training. However, appending is still the more optimal choice. This indicates that pause-pretraining induces considerable biases in how readily the delays are used based on their positional embeddings.

6 DISCUSSION AND KEY OPEN QUESTIONS

Enhanced computational width. One hypothesis as to why Transformer delays can help is that it increases expressivity by increasing the computational *width*. To produce the $(K+1)^{\text{th}}$ token, standard inference involves a computational depth of L (corresponding to the *sequential* computation of L layers), and a computational width of K (corresponding to the *parallel* K computations per layer). With M <pause> tokens however, we perform $K+M$ parallel computations. We hypothesize that this additional width helps certain downstream tasks. Take for example, comprehension-based question-answering tasks. Here, having a greater number of attention units per layer, would permit a finer distribution of attention across various parts of the supporting context (where the answer resides). We speculate that this would allow the lower layers to extract more precise and diverse representations, which a higher layer can more reliably aggregate to produce a final answer.

We formalize this in Theorem J.1 (stated informally below). Our key theoretical insight is that the attention module can have a high "raw" parameter-count-based capacity, but low "implementation capacity": the number of operations implemented for a given input, which is bottlenecked by the number of tokens. Pause tokens can help relieve this bottleneck and tap into the raw representational capacity. We hope this preliminary result can inspire further discussions on how to formalize the Transformer's implementation capacity and differentiate it from the raw parameter count.

Theorem 6.1. *(informal; see Theorem J.1) Assume that the attention module has sufficiently many parameters (K) that it is much larger than the number of inputs tokens (L). Then there are tasks that involve many independent computations N, where $N > L$ (but $N < K$), such that a 2-layer Transformer can implement the task if and only if it uses pause tokens.*

Computational expansion along with parameter expansion. How do the gains offered by <pause> tokens vary with parameter count? An immediate hypothesis would be that for smaller models, delays become more beneficial as they provide a much-needed capacity increase in an otherwise capacity-deprived model. But a preliminary comparison between our two model sizes surprisingly suggests the opposite. Intuitively, we conjecture that a smaller model may simply not have enough raw capacity to implement a variety of distinct computations to utilize the new pathways introduced by <pause> tokens. This intuition is also echoed in Theorem 6.1 where smaller models would break our assumption that parameter count K is large enough.

Computational expansion vs parameter expansion. There are trivial ways to extend the next-token computation process, say via more attention heads or more layers. For a fixed inference-time FLOPS budget, can these give similar gains as pause tokens? In Appendix I, we argue why this is not the case — attaining similar gains requires significant parameter expansion and a significant expansion of the FLOPS budget. Thus, it is both practically and theoretically remarkable that pause tokens can yield gains despite negligible addition to the parameter count.

Pause-inference vs. Chain-of-Thought. It is worth contrasting the above computational advantage with that enjoyed by chain-of-thought (CoT) prompting (Wei et al., 2022). Here, one prompts the model to generate a long series of intermediate reasoning steps before producing the final answer. Thus, CoT also corresponds to greater computational width, by way of delaying its final answer

(albeit with meaningful tokens). However, CoT has a vital added advantage: it also increases the computational depth to a significant extent (Feng et al., 2023, Theorem 3.3 and 3.4). In particular, each (meaningful) delay token generated by CoT is autoregressively generated by the model. Thus, if there are M such tokens and L layers, the final token arises out of roughly $M \cdot L$ sequentially composed operations. Thus, CoT has a computational depth that is larger by a *multiplicative* factor of M, compared to pause-inference. Finally, we note that one major advantange of CoT is that it does not seem to require any special modifications in the pretraining objective.

7 Related Work

MEMORY TOKENS Multiple works have proposed using dummy tokens as a ways of introducing memory into the model. Closest to our work is Burtsev et al. (2020) who *prepend* these tokens (rather than append them) and crucially, introduce them only during training and inference on the target tasks. On smaller scratch-trained models (with parameter counts of 10M, 65M and 277M) and a pretrained BERT model (109M), this reportedly gives minimal gains. This echoes our own mixed results for the comparable `StdPT_PauseFT` variant, and the fact that our smaller model shows gains on fewer datasets. Orthogonally, Sukhbaatar et al. (2019) introduce a large set of "global" memory tokens (of the order of ten million new parameters) across all layers. However, these are independent of the input, and only act as keys and values (not as queries). For vision transformers (ViTs), AdaTape (Xue et al., 2023) appends tokens from a learnt memory bank. Concurrent work in ViTs (Darcet et al., 2023), use multiple different "register tokens" appended to the image patches. In the context of recurrent networks, closest to us is Herel & Mikolov (2023) who train with "thinking" tokens for extra compute for the target task and find small perplexity gains on reasoning tasks. Grave et al. (2016) use a memory cache as a way to attend to past hidden vectors.

Adaptive Compute Transformers *Adaptive compute* methods provide users with flexibility in inference time depending on the input to the model. Although adaptive compute is not the objective of this paper (after all, we use the same number of pause tokens across all inputs), pause-inference can be viewed as a potential basis for a future adaptive compute method. The Universal Transformer (Dehghani et al., 2019) adaptively increases *serial* computations by repeating certain layers. Graves (2017) explored a similar recurrence over hidden units in the context of RNNs. A future adaptive-compute version of pause-inference would fall in a much less explored paradigm that varies the number of input tokens, akin to AdaTape in ViTs (Xue et al., 2023).

We discuss other relevant works in detail in Appendix K.

8 Conclusion, Limitations and Future Work

Pause-training takes a step beyond the paradigm of "immediate" next-token prediction in language models. The key idea is to train models with (dummy) `<pause>` tokens so that the model can learn to harness the additional inference-time computation. This can improve performance on a variety of tasks, *if* we train with `<pause>` tokens both during pretraining and downstream finetuning.

However, by extension of the fact that every downstream task has an optimal number of `<pause>` tokens, we do not claim that pause-training should benefit every downstream task. Some tasks may simply be better off with zero `<pause>` tokens. The most important limitation though is that the expense of pause-pretraining comes in the way of making this idea more widely accessible. Consequently, we do not study how the gains generalize across more model sizes (beyond 1B and 130M), or to encoder-decoder architectures, or to other pretraining mixtures and objectives. Next, while we have laid out some preliminary theory for why `<pause>` tokens may be beneficial, we leave a rigorous understanding for future study. We also leave open a variety of follow-up algorithmic questions: pause-training with multiple *different* `<pause>` tokens, better determining the number of `<pause>` tokens (perhaps using model confidence), inducing robustness to shifts in delays, and so on. But the most pressing next step would be to find ways to make delays helpful directly on a standard pretrained model. Overall, we hope that our work opens up many avenues for theoretical and practical work in the paradigm of delayed next-token prediction.

Acknowledgements: We would like to thank Kaifeng Lyu, Srinadh Bhojanapalli, Yongchao Zhou, Nikunj Saunshi, and Seungyeon Kim for helping set up the initial codebase and for their guidance.